This Week

Experiments

- grep
- qsort

Program 2

- z827

Class

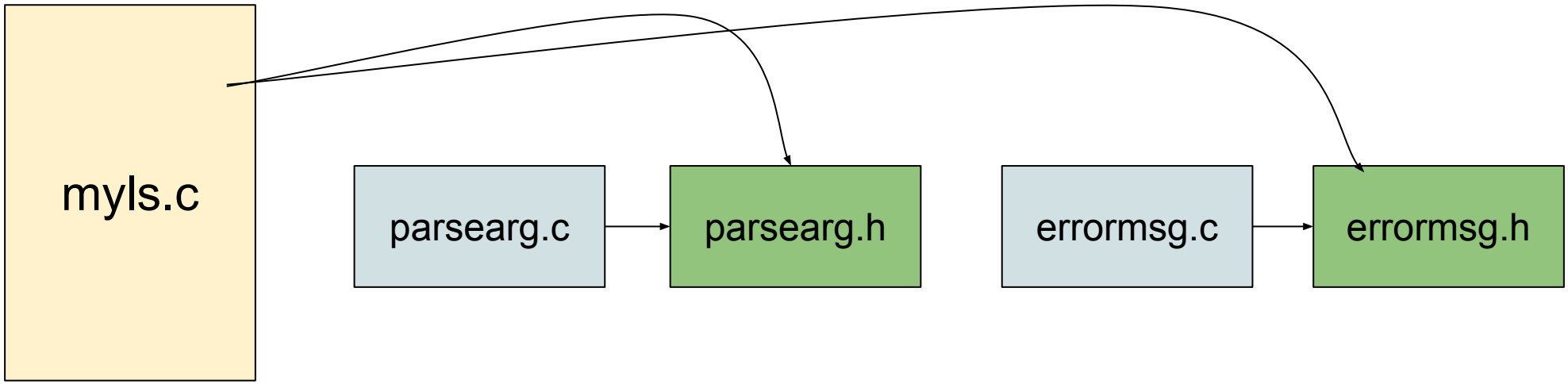
- make, malloc(), memory
- find, which, type, directories

COS 350 Module 09

unix: make

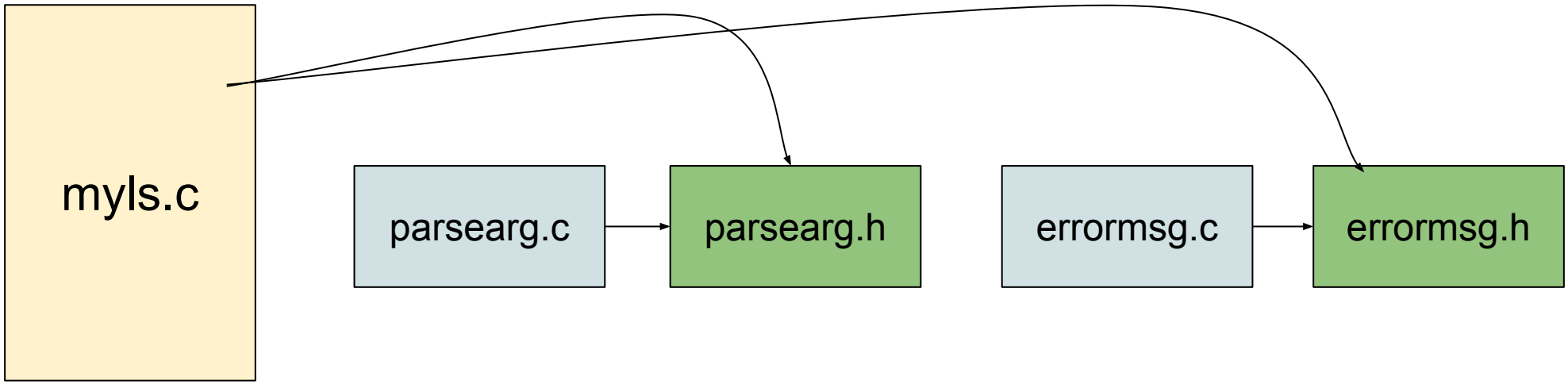
malloc

- strings
- arrays
- ragged arrays
- example: split



Complete compilation

gcc myls.c parsearg.c errormsg.c -o myls



Complete compilation

gcc myls.c parsearg.c errormsg.c -o myls

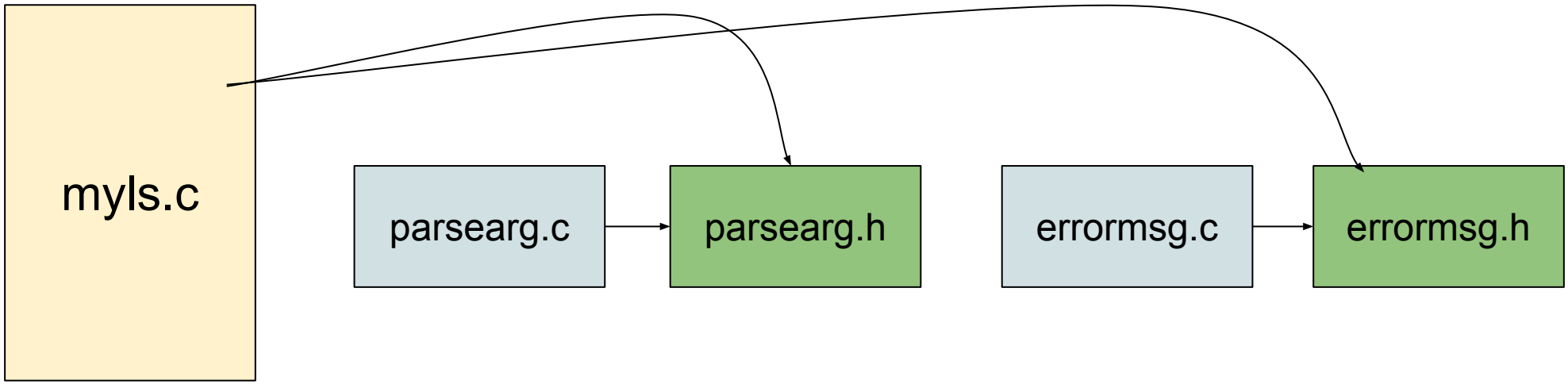
Separate compilation

gcc -c myls.c → myls.o

gcc -c parsearg.c → parsearg.o

gcc -c errormsg.c → errormsg.o

gcc myls.o parsearg.o errormsg.o -o myls




```
# makefile for myls.c

myls: myls.o parsearg.o errormsg.o
	gcc myls.o parsearg.o errormsg.o -o myls
myls.o: myls.c parsearg.h errormsg.h
	gcc -c myls.c
parsearg.o: parsearg.c parsearg.h
	gcc -c parsearg.c
errormsg.o: errormsg.c errormsg.h
	gcc -c errormsg.c
```

tab

Makefile

```
# makefile for myls.c

myls: myls.o parsearg.o errormsg.o
        gcc myls.o parsearg.o errormsg.o -o myls

myls.o: myls.c parsearg.h errormsg.h
        gcc -c myls.c

parsearg.o: parsearg.c parsearg.h
        gcc -c parsearg.c

errormsg.o: errormsg.c errormsg.h
        gcc -c errormsg.c
```

```
unix> make
gcc -c myls.c
gcc -c parsearg.c
gcc -c errormsg.c
gcc myls.o parsearg.o errormsg.o -o myls
```

Makefile

```
# makefile for myls.c
# with use of CFLAGS macro

CFLAGS = -g

myls: myls.o parsearg.o errormsg.o
   gcc $(CFLAGS) myls.o parsearg.o errormsg.o -o myls

myls.o: myls.c parsearg.c parsearg.h
   gcc -c $(CFLAGS) myls.c

parsearg.o: parsearg.c parsearg.h
   gcc -c $(CFLAGS) parsearg.c

errormsg.o: errormsg.c errormsg.h
   gcc -c $(CFLAGS) errormsg.c

clean:
   rm *.o
```

```
unix> make
gcc -c -g myls.c
gcc -c -g parsearg.c
gcc -c -g errormsg.c
gcc -g myls.o parsearg.o errormsg.o -o myls
```

Makefile

```
# makefile for myls.c
# with use of CFLAGS macro

CFLAGS = -g

myls: myls.o parsearg.o errormsg.o
   gcc $(CFLAGS) myls.o parsearg.o errormsg.o -o myls

myls.o: myls.c parsearg.c parsearg.h
   gcc -c $(CFLAGS) myls.c

parsearg.o: parsearg.c parsearg.h
   gcc -c $(CFLAGS) parsearg.c

errormsg.o: errormsg.c errormsg.h
   gcc -c $(CFLAGS) errormsg.c

clean:
   rm *.o
```

```
unix> make clean
rm *.o
```

malloc

(Memory Allocate)

NAME

malloc, free - allocate and free dynamic memory

SYNOPSIS

```
#include <stdlib.h>
void *malloc(size_t size);
void free(void *ptr);
```

DESCRIPTION

The malloc() function allocates size bytes and returns a pointer to the allocated memory. The memory is not initialized.

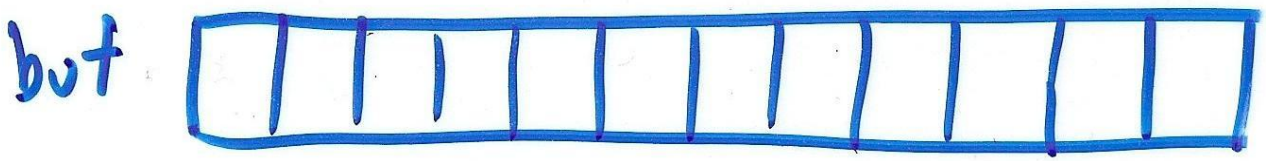
The free() function frees the memory space pointed to by ptr, which must have been returned by a previous call to malloc().

RETURN VALUE

The malloc() functions returns a pointer to the allocated memory, which is suitably aligned for any built-in type. On error it returns NULL.

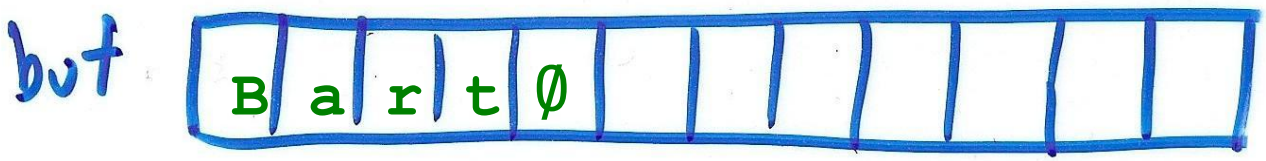
Example 1: Allocating a string buffer

```
char *s1 = "Bart";
char *s2 = "Simpson";
char *buf = malloc(strlen(s1)+strlen(s2)+1);
strcpy(buf, s1);
strcat(buf, s2);
```



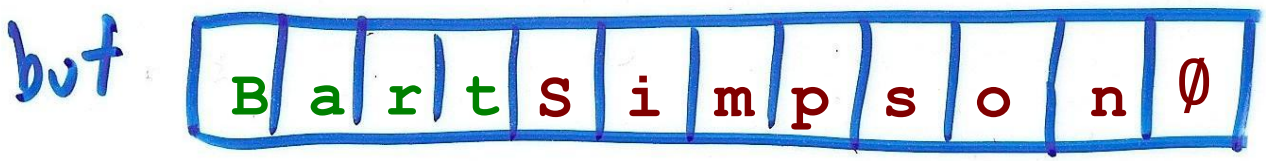
Example 1: Allocating a string buffer

```
char *s1 = "Bart";
char *s2 = "Simpson";
char *buf = malloc(strlen(s1)+strlen(s2)+1);
strcpy(buf, s1);
strcat(buf, s2);
```



Example 1: Allocating a string buffer

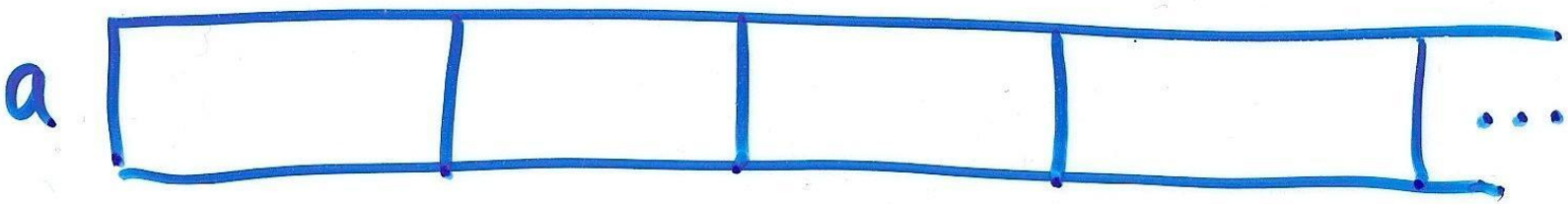
```
char *s1 = "Bart";
char *s2 = "Simpson";
char *buf = malloc(strlen(s1)+strlen(s2)+1);
strcpy(buf, s1);
strcat(buf, s2);
```




```
free(buf);
```

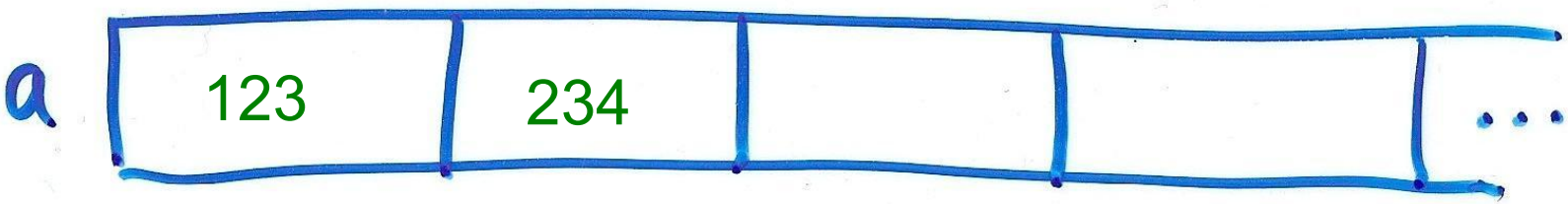
Example 2: Allocating a dynamic array

```
int n = 250;
int *a = malloc(n * sizeof(int));
```



Example 2: Allocating a dynamic array

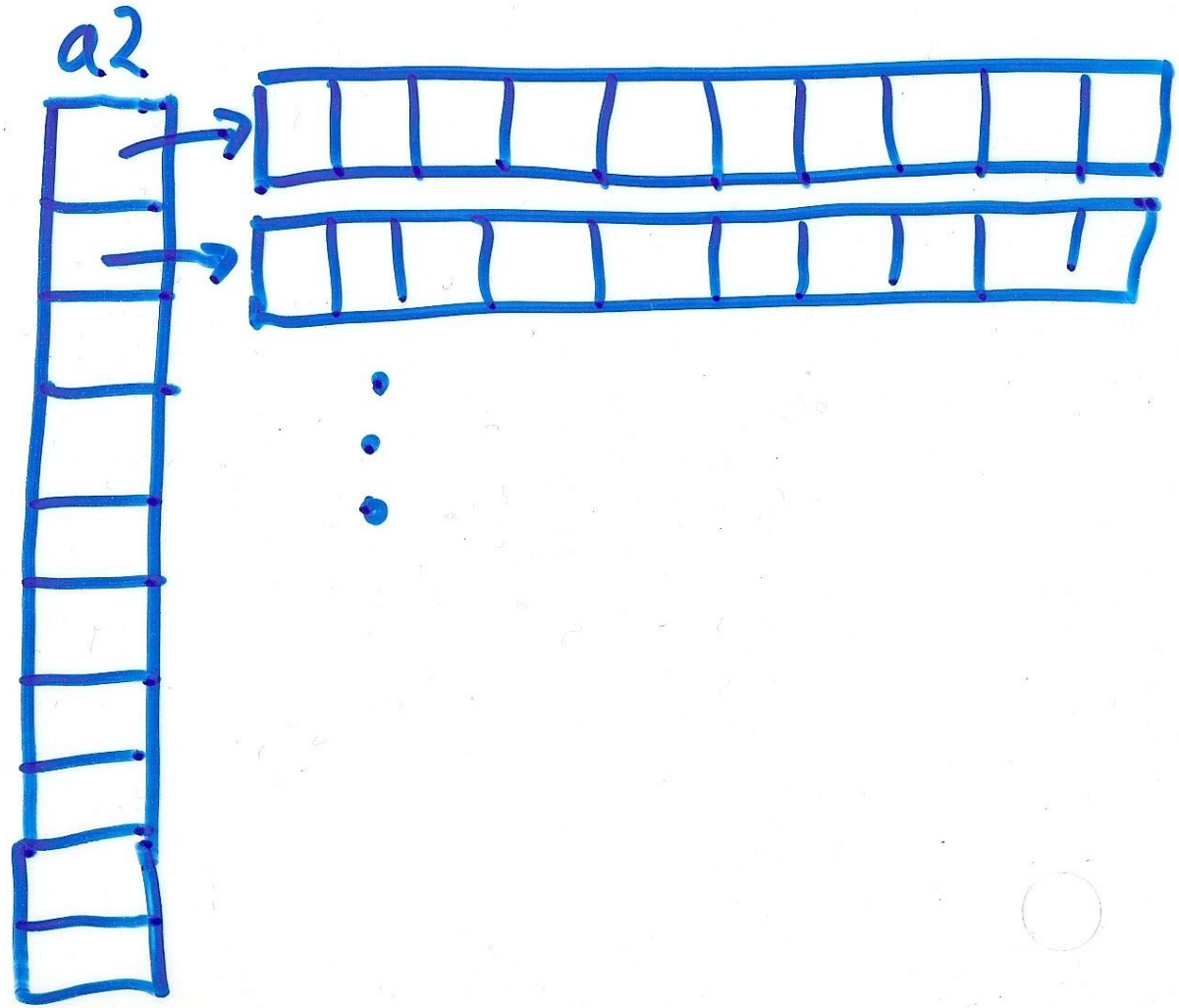
```
int n = 250;
int *a = malloc(n * sizeof(int));

a[0] = 123;
a[1] = 234;
```




```
free(a);
```

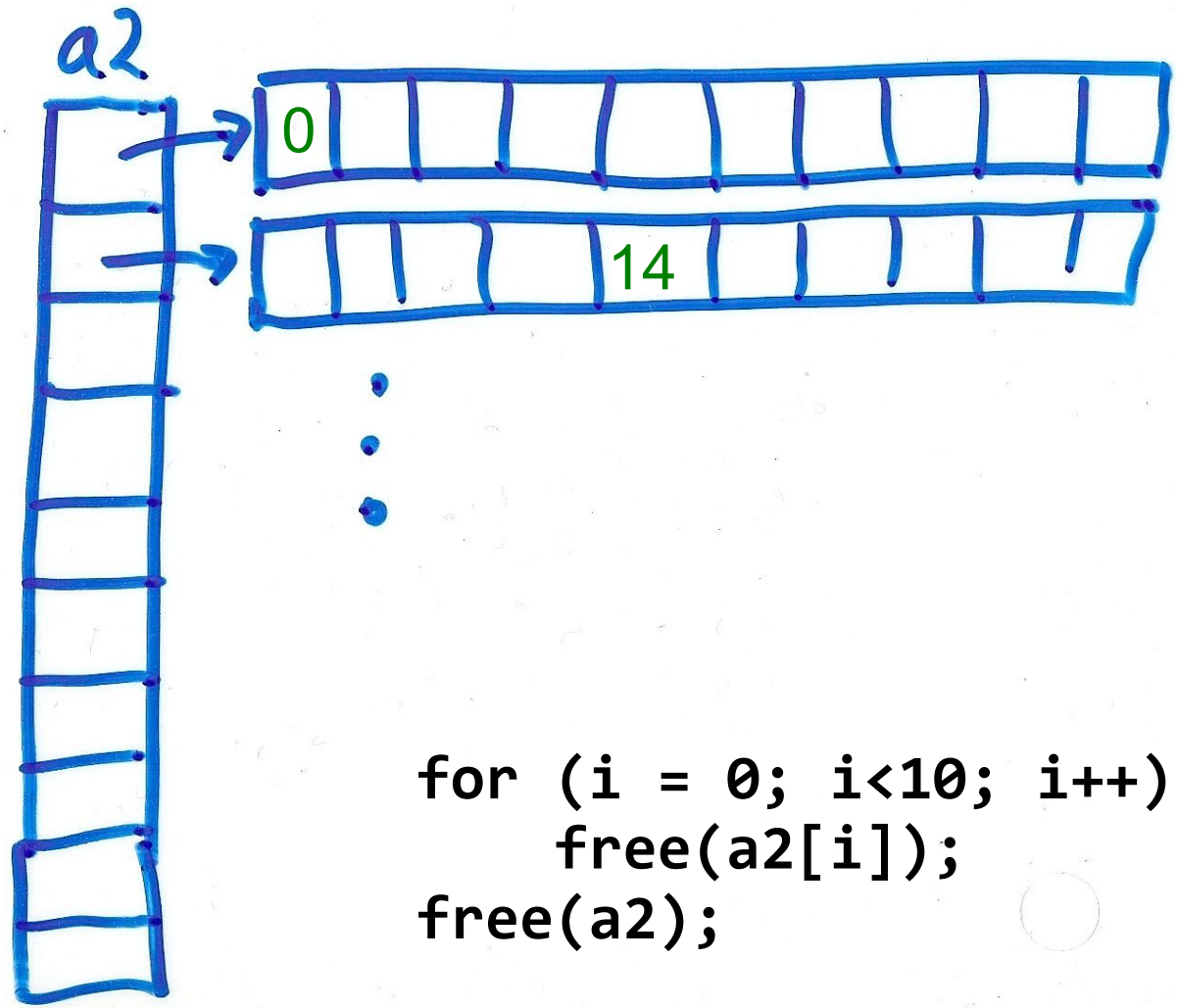
Example 3: Allocating a 10x10 ragged array

```
int **a2 = malloc(10*sizeof(int*));
int i;
for (i = 0; i<10; i++)
    a2[i] = malloc(10*sizeof(int));
```



Example 3: Allocating a 10x10 ragged array

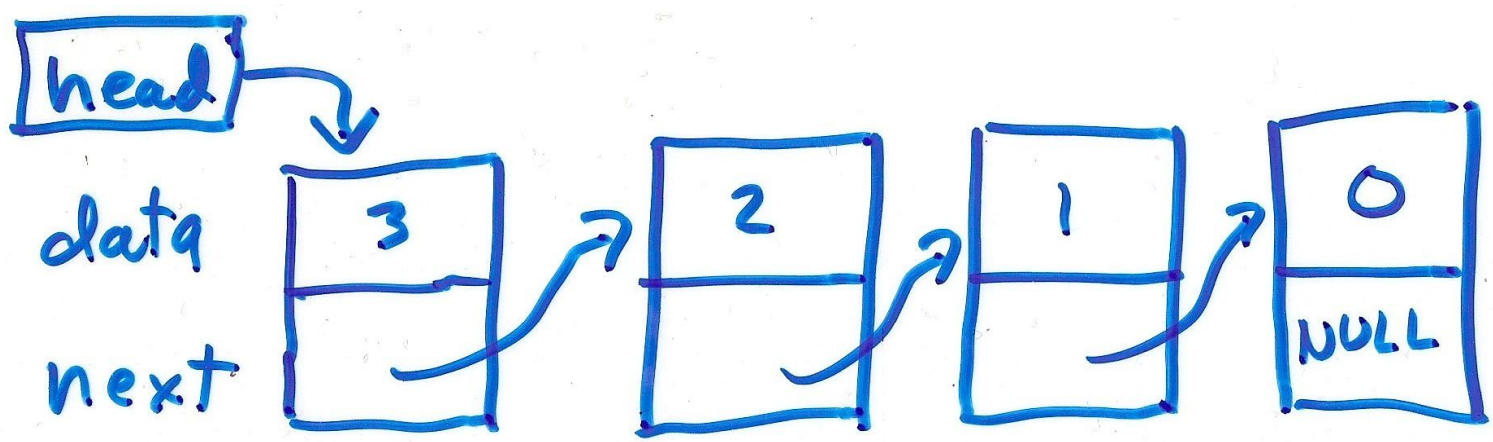
```
int **a2 = malloc(10*sizeof(int*));
int i;
for (i = 0; i<10; i++)
    a2[i] = malloc(10*sizeof(int));

a2[0][0] = 0;
a2[1][4] = 14;
```



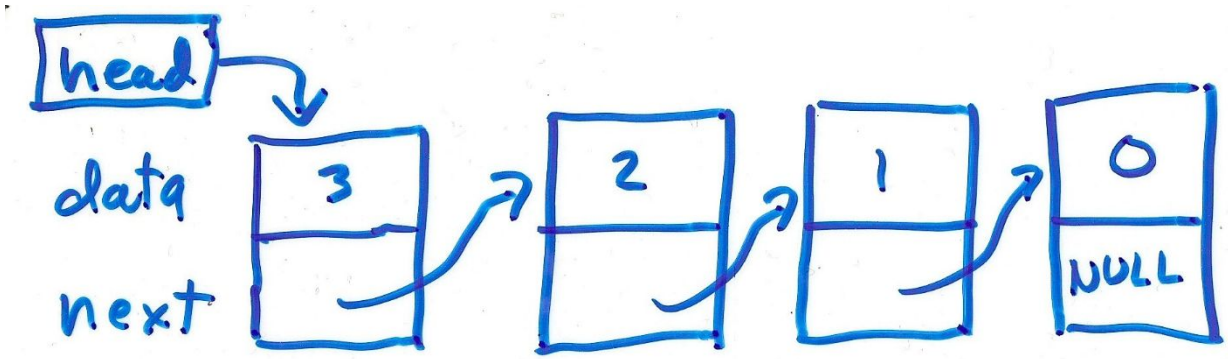

```
for (i = 0; i<10; i++)
    free(a2[i]);
free(a2);
```

Example 4: Allocating a linked list

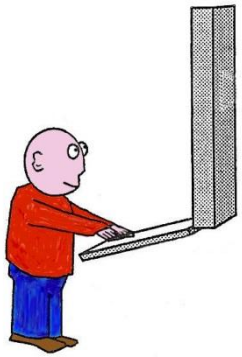
```
struct list {
   int data;
   struct list *next;
};

struct list *head = NULL;
for (i = 0; i < 1000; i++) {
    struct list *temp = malloc(sizeof(struct list));
    temp->data = i;
    temp->next = head;
    head = temp;
}
```



```
struct list {
   int data;
   struct list *next;
};

struct list *head = NULL;
for (i = 0; i < 1000; i++) {
    struct list *temp = malloc(sizeof(struct list));
    temp->data = i;
    temp->next = head;
    head = temp;
}
```




```
while (head) {
    struct list *temp = head;
    head = temp->next;
    free(temp);
}
```



split()
split2()

```
$ od -t x1 simple.txt
0000000 41 20 73 69 6d 70 6c 65 20 74 65 78 74 20 66 69
0000020 6c 65 0a

$ z827 simple.txt   # compress, remove original
…
$ od -t x1 simple.txt.z827
0000000 13 00 00 00 41 d0 3c dd 86 b3 cb 20 7a 19 4f 07
0000020 99 d3 ec b2 02
```

Header, 32-bit file size

Compressed file data

```
$ z827 simple.txt.z827  # decompress, remove .z827
…
```

A simple text file

z827

ASCII Hex values

A	“ “	s	i	m	p	l	e
0x41	0x20	0x73	0x69	0x6d	0x70	0x6c	0x65

0x41 is binary <u>0</u>100 0001
0x20 is binary <u>0</u>010 000**0**
0x73 is binary <u>0</u>111 00**11**

<u>Underlined</u> bits are compressed away (not needed)

Compressed

0100 0001 **11**01 0000 0011 1100

0x41 0xd0 0x3c

Some ideas to get you started

Compress 1 byte at a time:
Keep an unsigned int that you accumulate bits into.
Write out each byte and shift accordingly

→ 0000 0000 0100 0001 # first byte
→ 00010 000 0100 0001 # second byte (write)
→ 0000 0000 0001 0000 # shift over, ready for 3
→ 0000 1110 1101 0000 # second byte (write)

Track how many bits to shift
and position next byte, note that it changes

Block Compression:

Every block of 8 bytes compresses to a block of 7 bytes. You can compress a block at a time for speed. (<u>underlined bits are not used/compressed</u>)

0100 0001
0010 0000
0111 0011
0100 0001
0010 0000
0111 0011
0100 0001
0010 0000

0100 0001
11010 000
0011 1100
0000 0100
1001 1000
0000 0111
0100 0001